FPGA Tactical Controller for StarCraft II Micro Scenarios

Target Tier: Tier A (100%) **Course:** EECS 3216 (Fall 2026) **Topic:** Datapath / Control / VGA

Group Members: Marlin Lachance (213885025), Vaughn Chan (219846914), Hafsa Manazir (219780428), Vladislav Prasko (220741781)

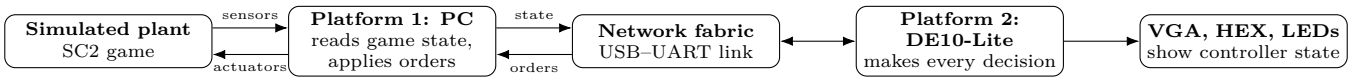
[built] working on the board; our group demonstrated it to the TA during scoping. [proposed] new work for the final demonstration.

1. Overview and Difficulty Justification

The FPGA commands Marines against waves of Roaches in StarCraft II (SC2); the PC only relays game state and orders.

- [built] On DeepMind's *DefeatRoaches* mini-game [1], the FPGA chooses which Roach each Marine attacks, and VGA draws the units and target lines.
- [proposed] In our own variant, the Marines start as two separate squads. Using Lanchester's square law, the FPGA decides when a squad is too weak to fight alone, regroups it with the other squad, and reengages together.

Tier A: a custom UART link, pipelined targeting datapath, control FSMs and VGA display run on the board at once.



The structure follows Lee and Seshia's cyber-physical system, Fig. 1.1 [2], with the SC2 game in place of a physical plant.

2. Target Hardware and Peripherals (DE10-Lite Only)

- **Onboard** (final layout): MAX 10 FPGA (50 MHz), GPIO header (UART), VGA port; each control is used in a Section 5 test:
  - **Switches:** SW0 chooses the targeting policy; SW1 turns regrouping on or off for the comparison runs.
  - **Keys:** KEY0 resets squad states and error flags before each run; KEY1 pauses the game (the board holds its reply).
  - **Displays:** HEX3:0 clock cycles per decision; HEX5:4 last error code; LEDR0/LEDR1 packet received/rejected.
- **External:** 3.3 V USB-to-UART adapter; VGA monitor; PC running SC2 with our Python bridge.

3. Combinational and Sequential Logic Architecture

- **Combinational:**
  - [built] Squared Marine–Roach distances, attack-range checks, target-ranking comparators, packet checksum.
  - [proposed] Squad strengths $P = kN^2$ (square lookup table and multiplier) and strength comparators.
- **Sequential:**
  - [built] UART FSMs; snapshot registers; ranking pipeline (one Marine–Roach pair per clock); VGA sync counters.
  - [proposed] A state machine for each squad that switches it between attacking, regrouping and holding.

4. Proposed Extension: Lanchester Regroup-and-Reengage

Lanchester's square law gives a force of N living units the strength $P = kN^2$ [4], with k fixed per unit type. Two 5-Marine squads apart total $2 \cdot 5^2k = 50k$, but $10^2k = 100k$ together, so the controller avoids being defeated one squad at a time.

- **Attack:** the squad fights while its own strength is at least the enemy's.
- **Regroup:** when a squad alone is weaker than the enemy, it disengages and moves to the other squad.
- **Reengage:** once together, they attack if combined strength beats the enemy's, else hold back (a margin prevents flicker).
- **Orders:** the board also sends each squad's state and rally point; squads are together within a set distance.
- **Success:** the weak squad arrives with survivors and the combined force wins the wave, vs. regrouping off.
- **Fallback:** if regrouping is unreliable by the demo, a weaker squad retreats to its start point instead.

5. Concrete Specifications and Testable Features

- **SC2 targeting and tactical decisions**
  - [built] SW0=0: nearest Roach. SW0=1: focus fire [3], the weakest Roach in range, otherwise the nearest.
  - [proposed] Each squad attacks, regroups or holds as described in Section 4.
  - *Test:* decisions match a Python reference model; same-start fights with regrouping on and off (SW1).
- **VGA visualization of controller state**
  - [built] Units, Marine-to-target lines and the current policy.
  - [proposed] Strength bars for squad A, squad B and both together against the enemy's strength; Marines coloured by squad state.
  - *Test:* while paused (KEY1), bars match reported strengths and Marine colours match reported squad states.
- **UART communication**
  - [built] 115200-baud request/reply link with a checksum; bad packets are rejected with an error code.
  - *Test:* send a corrupted packet; LEDR1 lights, HEX5:4 show the error code, and the next packet is processed without a reset.
- **Originality:** HDL by the group, with Claude (Anthropic) as coding assistant and commit co-author; bridge: `burnysc2` [5]; map [1].

6. Verification Plan and Team Task Allocation

- **Marlin Lachance (lead):** existing design, architecture, squad state machines, SC2 integration.
- **Vaughn Chan (hardware):** board setup, UART and VGA testing, strength and state display.
- **Hafsa Manazir (documentation):** report, references, results.
- **Vladislav Prasko (demonstration):** two-squad scenario, integration testing, presentation.

We will also report Quartus timing and resource use and the clock cycles per decision (HEX3:0).

[1] O. Vinyals et al., "StarCraft II: A New Challenge for Reinforcement Learning," arXiv:1708.04782, 2017.
[2] E. A. Lee and S. A. Seshia, *Introduction to Embedded Systems: A Cyber-Physical Systems Approach*, 2nd ed., MIT Press, 2017, Fig. 1.1.
[3] D. Churchill and M. Buro, "Portfolio Greedy Search and Simulation for Large-Scale Combat in StarCraft," *IEEE CIG*, 2013.
[4] M. Stanescu, N. Barriga and M. Buro, "Using Lanchester Attrition Laws for Combat Prediction in StarCraft," *AIIDE*, 2015.
[5] BurnySc2, `python-sc2` (PyPI package `burnysc2`, v7.3.0), `https://github.com/BurnySc2/python-sc2`.